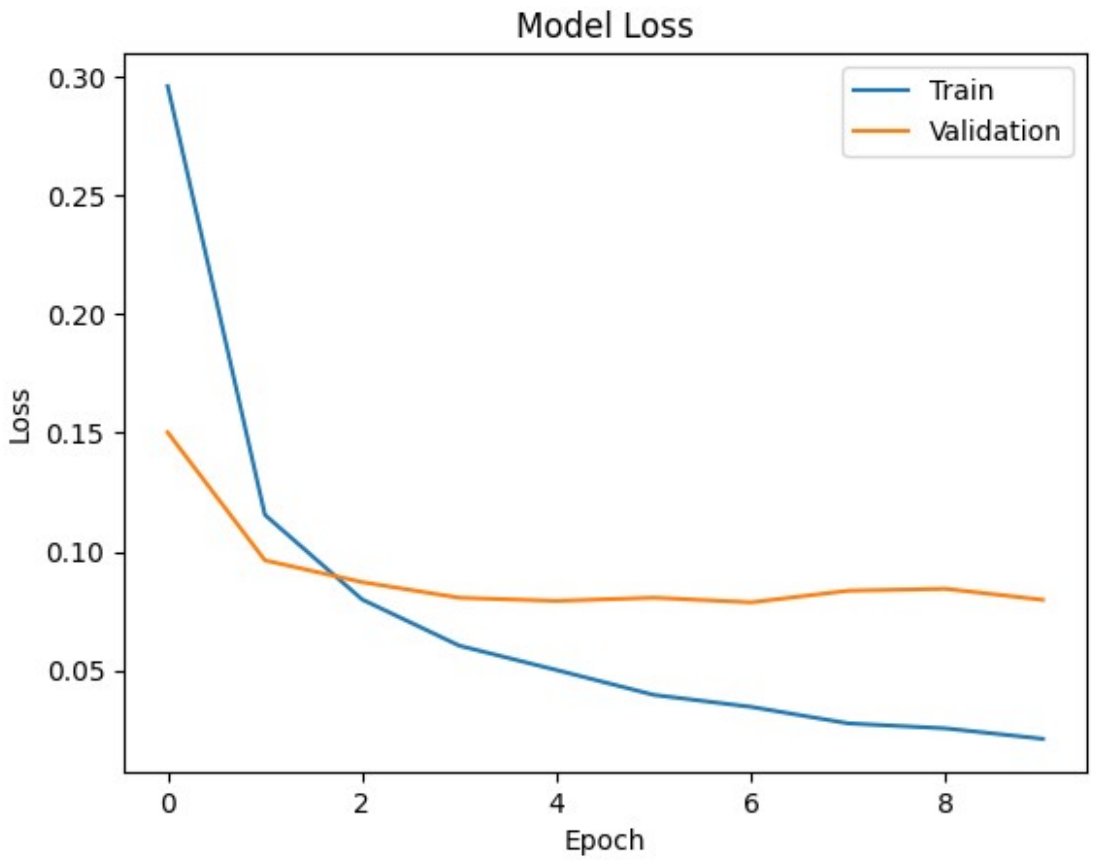


LAB 9 (19/03/2026)

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
Dense
from tensorflow.keras.models import Sequential
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0
# reshape for CNN
X_train_cnn = X_train.reshape(-1,28,28,1)
X_test_cnn = X_test.reshape(-1,28,28,1)

model = Sequential([
```

```
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),

    Flatten(),

    Dense(128, activation='relu'),
    Dense(10, activation='softmax')

])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train_cnn, y_train,
          epochs=10,
          batch_size=64,
          validation_split=0.1)

test_loss, test_acc = model.evaluate(X_test_cnn, y_test)

print("Test accuracy:", test_acc)
```

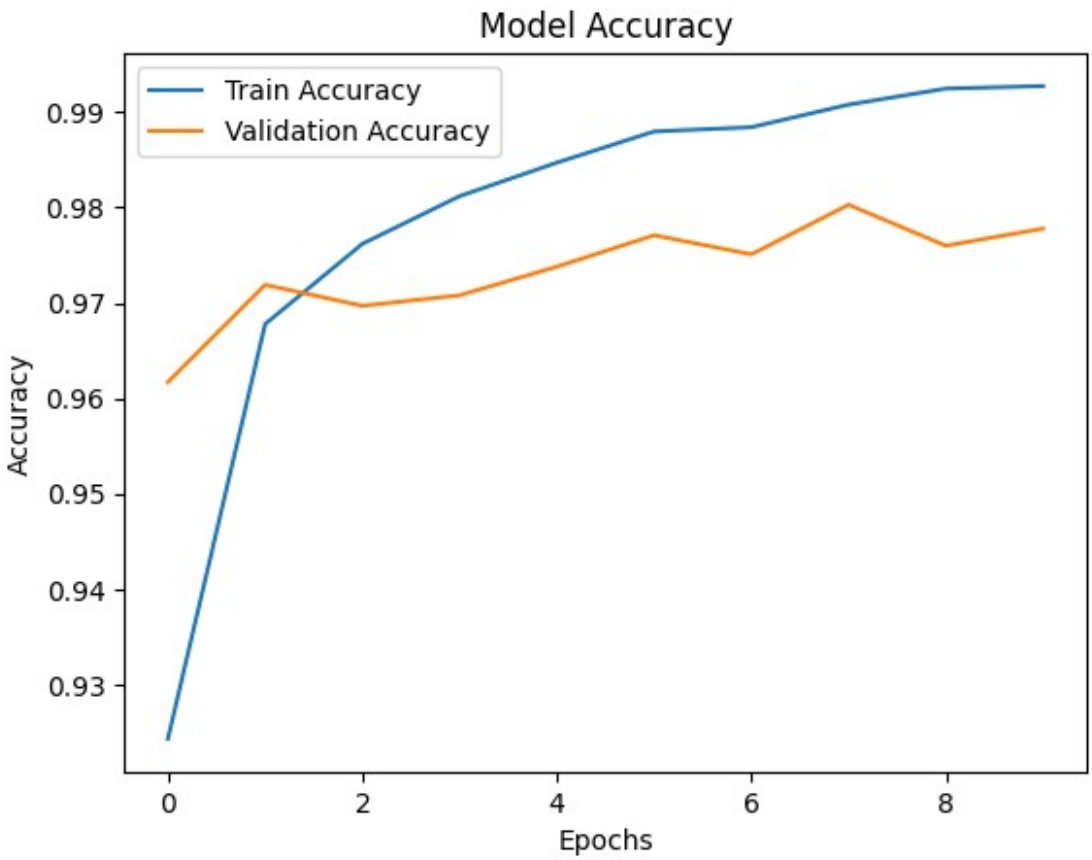
```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/
convolutional/base_conv.py:113: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

Model: "sequential_1"
```

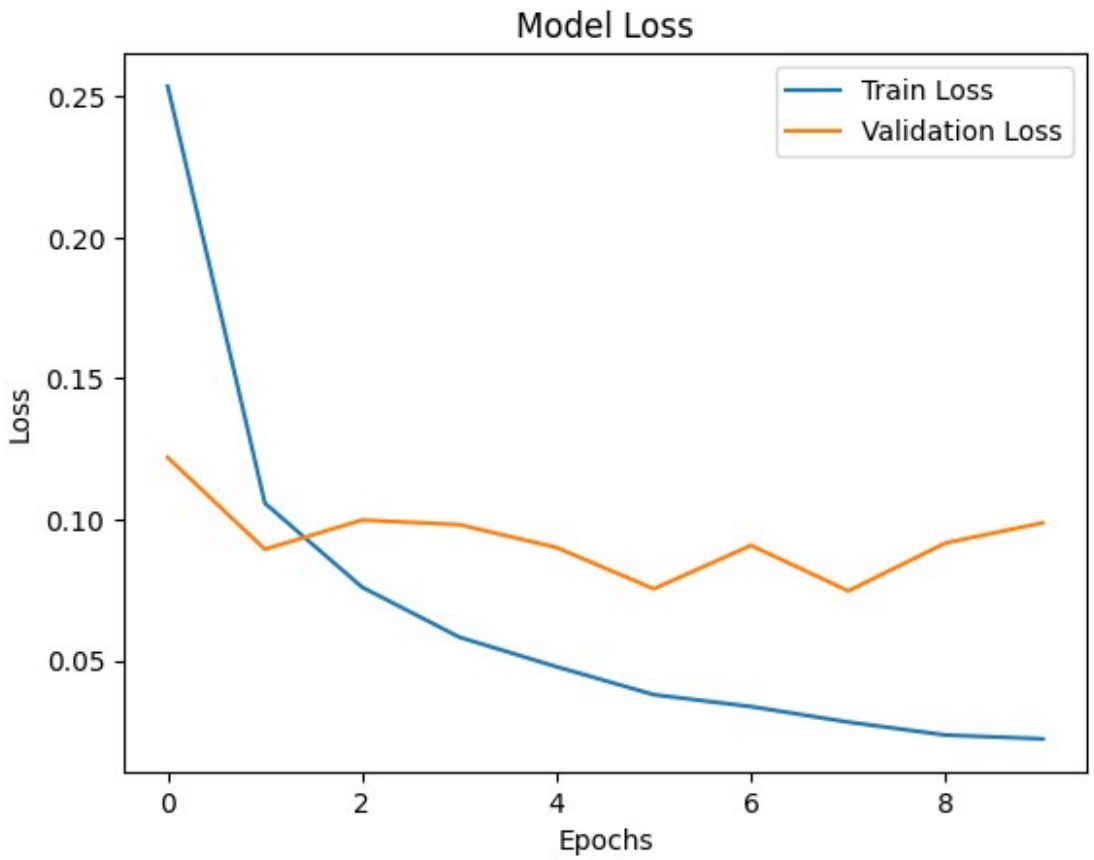
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0

```
├──────────────────────────────────┼─────────────────────────┼────────────
───────┤
│ conv2d_1 (Conv2D)                │ (None, 11, 11, 64)      │
18,496 │
├──────────────────────────────────┼─────────────────────────┼────────────
───────┤
│ max_pooling2d_1 (MaxPooling2D)   │ (None, 5, 5, 64)        │
0 │
├──────────────────────────────────┼─────────────────────────┼────────────
───────┤
│ flatten (Flatten)                │ (None, 1600)            │
0 │
├──────────────────────────────────┼─────────────────────────┼────────────
───────┤
│ dense_4 (Dense)                  │ (None, 128)             │
204,928 │
├──────────────────────────────────┼─────────────────────────┼────────────
───────┤
│ dense_5 (Dense)                  │ (None, 10)              │
1,290 │
└──────────────────────────────────┴─────────────────────────┴────────────
───────┘

 Total params: 225,034 (879.04 KB)

 Trainable params: 225,034 (879.04 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 61s 68ms/step - accuracy: 0.9478 - loss:
0.1724 - val_accuracy: 0.9830 - val_loss: 0.0570
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 53s 62ms/step - accuracy: 0.9852 - loss:
0.0491 - val_accuracy: 0.9890 - val_loss: 0.0382
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 52s 61ms/step - accuracy: 0.9896 - loss:
0.0337 - val_accuracy: 0.9875 - val_loss: 0.0373
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 59s 70ms/step - accuracy: 0.9927 - loss:
0.0238 - val_accuracy: 0.9903 - val_loss: 0.0388
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 75s 62ms/step - accuracy: 0.9939 - loss:
0.0187 - val_accuracy: 0.9917 - val_loss: 0.0363
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 54s 64ms/step - accuracy: 0.9951 - loss:
0.0153 - val_accuracy: 0.9917 - val_loss: 0.0327
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 55s 65ms/step - accuracy: 0.9963 - loss:
0.0113 - val_accuracy: 0.9920 - val_loss: 0.0318
```

```
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 79s 61ms/step - accuracy: 0.9969 - loss:
0.0091 - val_accuracy: 0.9922 - val_loss: 0.0366
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 53s 63ms/step - accuracy: 0.9969 - loss:
0.0080 - val_accuracy: 0.9943 - val_loss: 0.0323
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 53s 63ms/step - accuracy: 0.9981 - loss:
0.0060 - val_accuracy: 0.9935 - val_loss: 0.0346
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.9905 - loss:
0.0364
Test accuracy: 0.9904999732971191
```

```python
import matplotlib.pyplot as plt

# Accuracy Graph
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(['Train Accuracy', 'Validation Accuracy'])
plt.show()

# Loss Graph
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend(['Train Loss', 'Validation Loss'])
plt.show()
```


Model Accuracy
Train Accuracy
Validation Accuracy
Accuracy
0.99
0.98
0.97
0.96
0.95
0.94
0.93
0
2
4
6
8
Epochs




```
# 1. Import Libraries
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
Dense
import matplotlib.pyplot as plt
import numpy as np

# 2. Load Dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 3. Normalization
X_train = X_train / 255.0
X_test = X_test / 255.0

# 4. Train/Test already split in MNIST

# 5. Reshape for CNN (important)
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

# 6. Build Model (Conv → Pool → Flatten → Dense)
```

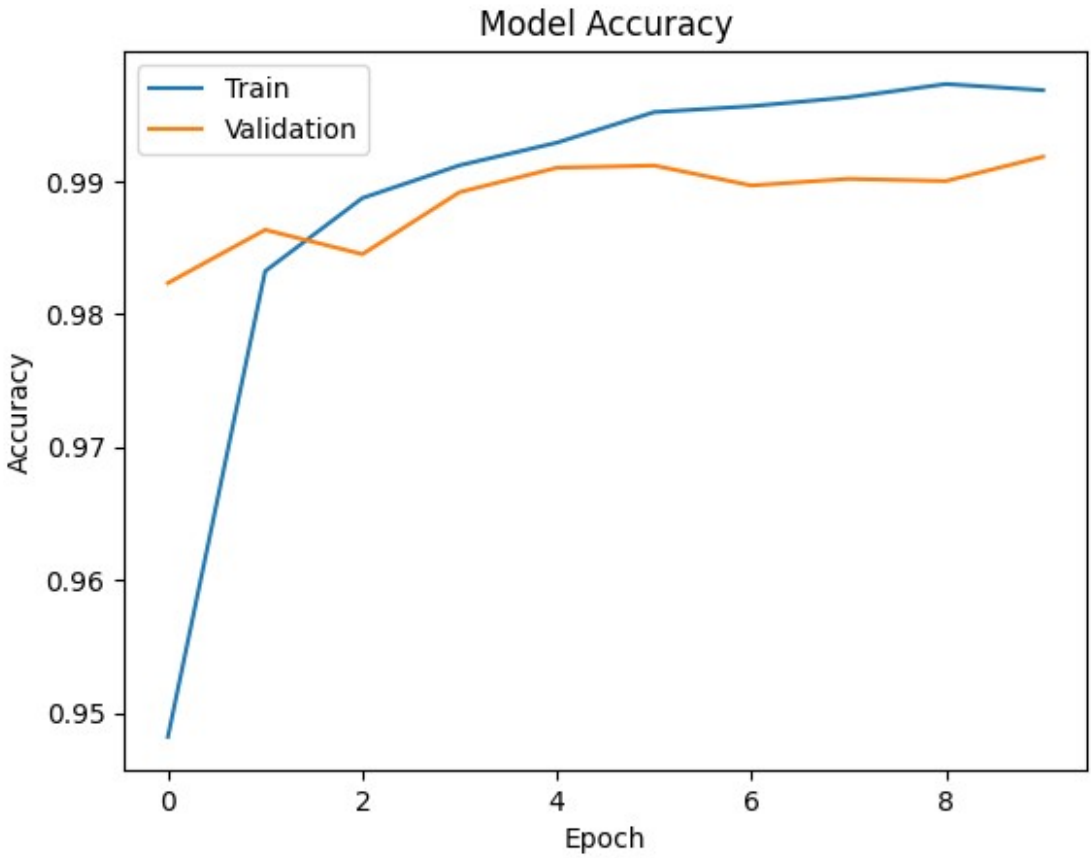
```
model = Sequential()

# Convolution
model.add(Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))

# Pooling
model.add(MaxPooling2D((2,2)))

# Convolution
model.add(Conv2D(64, (3,3), activation='relu'))

# Pooling
model.add(MaxPooling2D((2,2)))

# Flatten
model.add(Flatten())

# Dense
model.add(Dense(128, activation='relu'))

# Output Dense
model.add(Dense(10, activation='softmax'))

# 7. Compile
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# 8. Summary
model.summary()

# 9. Training
history = model.fit(X_train, y_train,
                    epochs=10,
                    batch_size=64,
                    validation_split=0.1)

# 10. Evaluation
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)

# 11. Graphs (Loss & Accuracy)

# Accuracy Graph
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
```

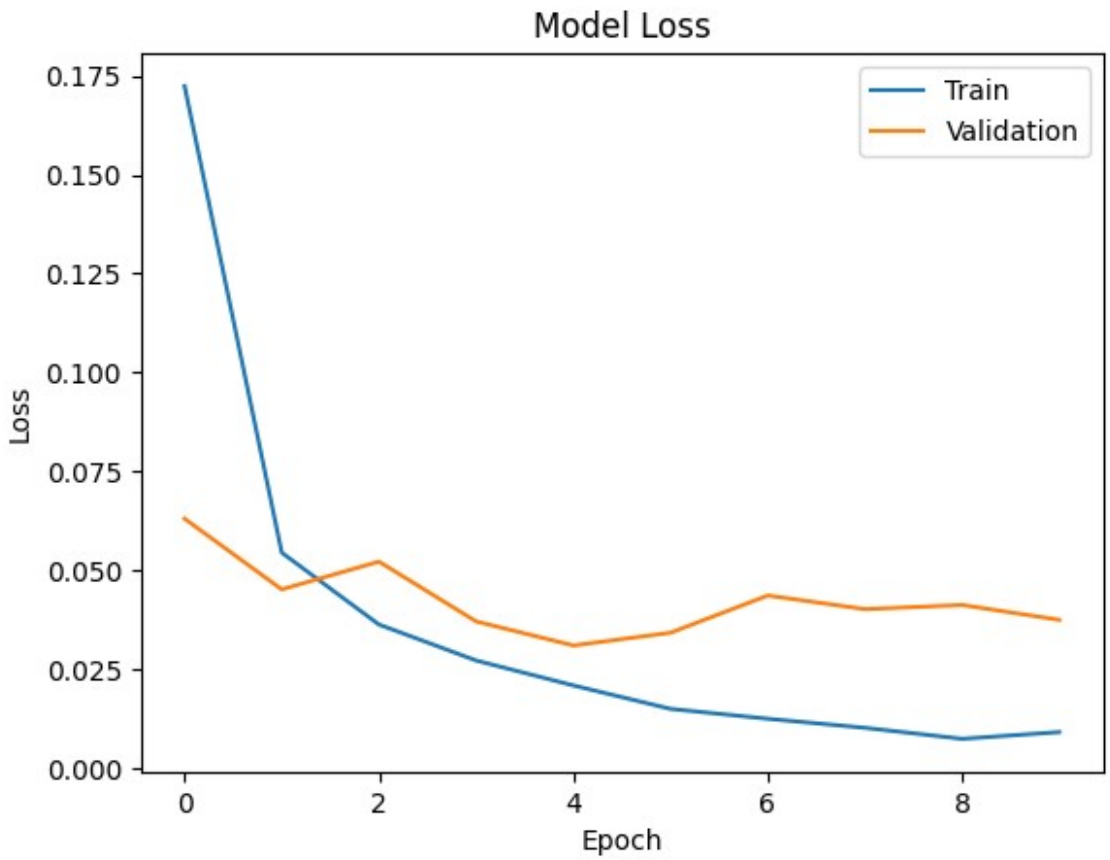
```
plt.legend(['Train', 'Validation'])
plt.show()

# Loss Graph
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(['Train', 'Validation'])
plt.show()
```

```
Model: "sequential_2"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape           ┃       Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ conv2d_2 (Conv2D)               │ (None, 26, 26, 32)     │           320 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ max_pooling2d_2 (MaxPooling2D)  │ (None, 13, 13, 32)     │             0 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ conv2d_3 (Conv2D)               │ (None, 11, 11, 64)     │        18,496 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ max_pooling2d_3 (MaxPooling2D)  │ (None, 5, 5, 64)       │             0 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ flatten_1 (Flatten)             │ (None, 1600)           │             0 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense_6 (Dense)                 │ (None, 128)            │       204,928 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense_7 (Dense)                 │ (None, 10)             │         1,290 │
└─────────────────────────────────┴────────────────────────┴───────────────┘
```

```
 Total params: 225,034 (879.04 KB)

 Trainable params: 225,034 (879.04 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 63s 71ms/step - accuracy: 0.9482 - loss:
0.1724 - val_accuracy: 0.9823 - val_loss: 0.0631
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 51s 60ms/step - accuracy: 0.9832 - loss:
0.0546 - val_accuracy: 0.9863 - val_loss: 0.0453
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 53s 62ms/step - accuracy: 0.9887 - loss:
0.0364 - val_accuracy: 0.9845 - val_loss: 0.0523
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 83s 64ms/step - accuracy: 0.9912 - loss:
0.0273 - val_accuracy: 0.9892 - val_loss: 0.0372
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 81s 62ms/step - accuracy: 0.9929 - loss:
0.0210 - val_accuracy: 0.9910 - val_loss: 0.0311
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 83s 64ms/step - accuracy: 0.9952 - loss:
0.0151 - val_accuracy: 0.9912 - val_loss: 0.0343
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 64ms/step - accuracy: 0.9956 - loss:
0.0126 - val_accuracy: 0.9897 - val_loss: 0.0437
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 52s 62ms/step - accuracy: 0.9963 - loss:
0.0103 - val_accuracy: 0.9902 - val_loss: 0.0403
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 53s 63ms/step - accuracy: 0.9973 - loss:
0.0075 - val_accuracy: 0.9900 - val_loss: 0.0413
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 63ms/step - accuracy: 0.9968 - loss:
0.0092 - val_accuracy: 0.9918 - val_loss: 0.0376
313/313 ━━━━━━━━━━━━━━━━━━━━ 5s 15ms/step - accuracy: 0.9895 - loss:
0.0393
Test Accuracy: 0.9894999861717224
```


Model Accuracy
Train
Validation
0.99
0.98
0.97
0.96
0.95
Accuracy
0
2
4
6
8
Epoch




```
import matplotlib.pyplot as plt

batch_sizes = [32, 64, 128]
histories = {}

for batch in batch_sizes:
    model = Sequential([
        Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
        MaxPooling2D((2,2)),

        Conv2D(64, (3,3), activation='relu'),
        MaxPooling2D((2,2)),

        Flatten(),
        Dense(128, activation='relu'),
        Dense(10, activation='softmax')
    ])

    model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

    print(f"\nTraining with batch size: {batch}")
```

```
    history = model.fit(X_train_cnn, y_train,
                        epochs=10,
                        batch_size=batch,
                        validation_split=0.1,
                        verbose=1)

    histories[batch] = history

# Plot accuracy
plt.figure(figsize=(8,5))

for batch in batch_sizes:
    plt.plot(histories[batch].history['accuracy'], label=f'Train Acc
(batch={batch})')
    plt.plot(histories[batch].history['val_accuracy'], linestyle='--',
label=f'Val Acc (batch={batch})')

plt.title("Accuracy vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()


Training with batch size: 32
Epoch 1/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 71s 39ms/step - accuracy: 0.9585 -
loss: 0.1375 - val_accuracy: 0.9870 - val_loss: 0.0475
Epoch 2/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 62s 37ms/step - accuracy: 0.9855 -
loss: 0.0449 - val_accuracy: 0.9883 - val_loss: 0.0444
Epoch 3/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 63s 38ms/step - accuracy: 0.9904 -
loss: 0.0310 - val_accuracy: 0.9912 - val_loss: 0.0348
Epoch 4/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 63s 37ms/step - accuracy: 0.9925 -
loss: 0.0222 - val_accuracy: 0.9898 - val_loss: 0.0413
Epoch 5/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 60s 35ms/step - accuracy: 0.9946 -
loss: 0.0158 - val_accuracy: 0.9900 - val_loss: 0.0340
Epoch 6/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 63s 37ms/step - accuracy: 0.9953 -
loss: 0.0133 - val_accuracy: 0.9913 - val_loss: 0.0399
Epoch 7/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 61s 36ms/step - accuracy: 0.9965 -
loss: 0.0099 - val_accuracy: 0.9910 - val_loss: 0.0359
Epoch 8/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 66s 39ms/step - accuracy: 0.9970 -
loss: 0.0088 - val_accuracy: 0.9908 - val_loss: 0.0441
```

```
Epoch 9/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 62s 37ms/step - accuracy: 0.9975 -
loss: 0.0073 - val_accuracy: 0.9922 - val_loss: 0.0364
Epoch 10/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 61s 36ms/step - accuracy: 0.9976 -
loss: 0.0072 - val_accuracy: 0.9917 - val_loss: 0.0425

Training with batch size: 64
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 61s 70ms/step - accuracy: 0.9468 - loss:
0.1734 - val_accuracy: 0.9847 - val_loss: 0.0523
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 54s 64ms/step - accuracy: 0.9839 - loss:
0.0512 - val_accuracy: 0.9857 - val_loss: 0.0476
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 54s 64ms/step - accuracy: 0.9888 - loss:
0.0354 - val_accuracy: 0.9903 - val_loss: 0.0376
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 55s 65ms/step - accuracy: 0.9917 - loss:
0.0261 - val_accuracy: 0.9873 - val_loss: 0.0393
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 55s 65ms/step - accuracy: 0.9932 - loss:
0.0208 - val_accuracy: 0.9907 - val_loss: 0.0344
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 83s 66ms/step - accuracy: 0.9949 - loss:
0.0162 - val_accuracy: 0.9900 - val_loss: 0.0392
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 56s 66ms/step - accuracy: 0.9956 - loss:
0.0127 - val_accuracy: 0.9927 - val_loss: 0.0309
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 56s 66ms/step - accuracy: 0.9968 - loss:
0.0100 - val_accuracy: 0.9912 - val_loss: 0.0376
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 55s 65ms/step - accuracy: 0.9972 - loss:
0.0086 - val_accuracy: 0.9903 - val_loss: 0.0366
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 65ms/step - accuracy: 0.9969 - loss:
0.0088 - val_accuracy: 0.9915 - val_loss: 0.0411

Training with batch size: 128
Epoch 1/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 51s 117ms/step - accuracy: 0.9356 - loss:
0.2175 - val_accuracy: 0.9808 - val_loss: 0.0679
Epoch 2/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 48s 113ms/step - accuracy: 0.9806 - loss:
0.0619 - val_accuracy: 0.9852 - val_loss: 0.0519
Epoch 3/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 49s 116ms/step - accuracy: 0.9870 - loss:
0.0428 - val_accuracy: 0.9882 - val_loss: 0.0424
```

```
Epoch 4/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 81s 113ms/step - accuracy: 0.9902 - loss:
0.0312 - val_accuracy: 0.9882 - val_loss: 0.0392
Epoch 5/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 81s 112ms/step - accuracy: 0.9918 - loss:
0.0255 - val_accuracy: 0.9900 - val_loss: 0.0376
Epoch 6/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 49s 116ms/step - accuracy: 0.9934 - loss:
0.0200 - val_accuracy: 0.9873 - val_loss: 0.0447
Epoch 7/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 81s 112ms/step - accuracy: 0.9947 - loss:
0.0159 - val_accuracy: 0.9893 - val_loss: 0.0408
Epoch 8/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 48s 115ms/step - accuracy: 0.9957 - loss:
0.0138 - val_accuracy: 0.9908 - val_loss: 0.0349
Epoch 9/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 83s 116ms/step - accuracy: 0.9970 - loss:
0.0096 - val_accuracy: 0.9892 - val_loss: 0.0432
Epoch 10/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 81s 114ms/step - accuracy: 0.9972 - loss:
0.0082 - val_accuracy: 0.9910 - val_loss: 0.0405
```